PERSISTENCIA EN JAVA 21 · PROGRAMA 5

Programa 5: Persistencia en archivos CSV con Java 21

Práctica de laboratorio · Estructura producto: código, nombre, cantidad, precio y total

Esta práctica amplía la serie de persistencia de productos con un quinto programa independiente. Implementa crear, añadir, listar, modificar y eliminar utilizando `productos.csv`, sin bibliotecas externas, y mantiene el menú y la organización por capas de los programas anteriores.

Objetivos de aprendizaje

- **Modelo de datos:** representar un producto mediante un `record` inmutable y validar sus atributos en el constructor.
- **Formato CSV:** distinguir los separadores de columna de las comas que pertenecen a un nombre.
- **Lectura y escritura:** conservar comillas, comas y saltos de línea dentro de los campos.
- **Operaciones CRUD:** cargar los productos, cambiar la colección en memoria y guardar el resultado.
- **Protección de datos:** escribir primero en un temporal y detener las operaciones cuando el archivo contenga errores.

Requisitos y archivos de la práctica

Se necesita un JDK 21 y una consola; las instrucciones se presentan para Windows 11. Este programa no necesita Gson, un driver JDBC ni una base de datos.

```
programa5-csv/
├── Programa5Csv.java            <- programa independiente
├── PruebasPrograma5Csv.java     <- pruebas automáticas
├── Programa5-CSV.md             <- guía de la práctica
├── Programa5-CSV.pdf            <- guía para los estudiantes
├── ejemplo-productos.csv        <- datos para importar
└── productos.csv               <- archivo creado al ejecutar
```

El archivo activo se llama `productos.csv` y se ubica en la carpeta desde la que se ejecuta el programa. El archivo de ejemplo tiene otro nombre para evitar cargar o sobrescribir datos sin que el estudiante lo decida.

Estructura producto y reglas de validación

Atributo	Tipo Java	Regla aplicada
`codigo`	`String`	Obligatorio, único y sin espacios, incluidos espacios Unicode. Se convierte a mayúsculas con `Locale.ROOT`.
`nombre`	`String`	Obligatorio; admite espacios, comas, comillas y punto y coma. Se quitan espacios sobrantes al inicio y al final.
`cantidad`	`int`	Entero mayor o igual a cero.
`precio`	`double`	Finito, mayor o igual a cero; se redondea a dos decimales.
`total()`	`double`	Resultado calculado de `cantidad * precio`; no se solicita al usuario.

El código identifica al producto y no se cambia durante una modificación. Si se necesita otro código, se elimina el producto y se añade uno nuevo.

El constructor rechaza `NaN`, infinito y combinaciones cuyo total desborde el rango de `double`. Para normalizar el precio se utiliza `BigDecimal.valueOf(precio).setScale(2, RoundingMode.HALF_UP).doubleValue()`; el atributo sigue siendo `double`, tal como se solicitó, por lo que la aritmética posterior conserva las limitaciones de precisión de ese tipo.

Ejemplo de cálculo

Para una cantidad de 5 y un precio de 9.90, el total es 49.50. Si la cantidad cambia a 8, el total pasa a 79.20 sin que el usuario tenga que introducirlo.

El archivo incluye una columna `total` para facilitar su lectura, pero el programa no utiliza ese valor al cargar. Siempre lo recalcula a partir de cantidad y precio.

Formato del archivo CSV

En esta práctica se utiliza la coma como separador, el punto como separador decimal y UTF-8 como codificación. La primera fila contiene exactamente los cinco nombres de columna, en el orden indicado.

```
codigo,nombre,cantidad,precio,total
"P001","Arroz extra superior",10,4.50,45.00
"P002","Aceite ""Premium"", botella; 1 L",5,9.90,49.50
"P003","Leche evaporada",12,3.80,45.60
```

Reglas utilizadas

- **Cabecera:** `codigo,nombre,cantidad,precio,total`; no lleva el prefijo `#` del programa de texto plano.
- **Campos de texto:** el escritor siempre encierra código y nombre entre comillas dobles, aunque no contengan comas.
- **Comillas interiores:** cada comilla del dato se convierte en dos comillas dentro del CSV.
- **Números:** se escriben sin comillas y con punto decimal; precio y total se muestran con dos decimales.
- **Final de registro:** el escritor utiliza CRLF (`\r\n`); el lector también acepta archivos con LF.
- **Saltos dentro de un campo:** el analizador los conserva cuando están dentro de comillas. La consola solicita nombres de una sola línea; los nombres multilínea se pueden cargar desde un CSV.

El lector admite campos de texto sin comillas si no contienen caracteres que lo impidan. También tolera un marcador BOM al principio del archivo importado; el escritor no lo añade.

Por qué no sirve usar split(",")

El nombre `Aceite "Premium", botella; 1 L` contiene una coma que pertenece al producto. Separar la línea con `split(",")` produciría más columnas de las esperadas y no interpretaría las comillas duplicadas.

```
Dato original:
Aceite "Premium", botella; 1 L

Representación CSV:
"Aceite ""Premium"", botella; 1 L"

Dato recuperado:
Aceite "Premium", botella; 1 L
```

El CSV también es texto plano; la diferencia frente al formato simplificado del programa 1 es que aquí se aplican reglas explícitas de entrecomillado y escape. No basta con cambiar la extensión de `.txt` a `.csv`.

Arquitectura del programa

Se conserva la separación por responsabilidades de los otros ejemplos. Todas las clases necesarias están dentro de `Programa5Csv.java`, por lo que no es necesario copiar clases de los programas 1 a 4.

```
Menú y métodos de interacción
            |
            v
Consola: entrada validada
            |
            v
ServicioProductos: reglas CRUD
            |
            v
RepositorioCsv: analizar, cargar y guardar
            |
            v
productos.csv

Producto: modelo validado utilizado por las capas
```

Modelo: record Producto

El `record` contiene los cuatro datos ingresados y el método `total()`. El constructor compacto se ejecuta tanto al crear un producto desde la consola como al reconstruirlo desde el archivo.

```java
record Producto(String codigo, String nombre, int cantidad, double precio) {
    // Constructor compacto: valida y normaliza los cuatro atributos.
    double total() {
        return cantidad * precio;
    }
}
```

El fragmento anterior es un esquema; el código completo aparece al final de esta guía. Para modificar datos no se alteran los atributos del objeto existente: se crea otro `Producto` con el mismo código y los nuevos valores.

Repositorio: RepositorioCsv

Método	Responsabilidad
`existe()`	Comprobar si existe el archivo.
`crear()`	Guardar una colección vacía, dejando únicamente la cabecera.
`cargarTodos()`	Leer el CSV, comprobar su estructura y devolver un mapa por código.
`guardarTodos(...)`	Escribir la colección completa en un temporal y reemplazar el archivo.
`escapar(...)`	Encerrar un texto entre comillas y duplicar sus comillas interiores.
`analizar(...)`	Separar registros y campos respetando el entrecomillado.

Analizador CSV: lectura carácter por carácter

El método `analizar` no divide el archivo por líneas antes de interpretar los campos. Recorre todo el texto y conserva el estado necesario para distinguir una coma separadora de una coma que forma parte del nombre.

Variable	Qué representa
`entreComillas`	El campo está abierto entre comillas; comas y saltos se guardan como datos.
`cerrado`	Ya apareció la comilla final; solo se acepta un separador o el final del archivo.
`iniciado`	El campo ya empezó; permite detectar comillas mal colocadas y campos vacíos.
`campo`	Los caracteres del atributo actual, acumulados en un `StringBuilder`.
`fila`	Los atributos que ya se completaron en el registro actual.
`filas`	Los registros completos del documento.

Dentro de un campo entrecomillado, dos comillas consecutivas representan una sola comilla del dato. Fuera de comillas, una coma completa el campo y un salto de línea completa el registro; un CRLF cuenta como un único final de registro.

Cuando termina el texto se comprueba que no haya comillas abiertas. También se evita crear un registro vacío ficticio por el salto de línea que normalmente aparece al final del archivo.

Carga y validación del documento

La carga exige una cabecera exacta y cinco columnas por registro. Después convierte cantidad y precio a sus tipos Java, crea el `Producto` y verifica que su código no esté repetido.

El total guardado se ignora, pero su columna debe estar presente. Una fila vacía intermedia, una cantidad negativa, un código duplicado o una estructura de comillas incorrecta detienen la carga con un mensaje de error.

Esta práctica no omite silenciosamente filas defectuosas. Si se permitiera ignorarlas y luego se guardara solo lo que se pudo cargar, se perderían las filas originales en la siguiente modificación.

Escritura y sustitución del archivo

El escritor crea un temporal único en la misma carpeta de destino. Escribe la cabecera y todos los productos, cierra el flujo y solicita el reemplazo con `ATOMIC_MOVE`.

Si el sistema no admite ese movimiento, se utiliza `REPLACE_EXISTING` sin la opción atómica. Ese respaldo permite funcionar en más entornos, pero no ofrece la misma garantía frente a una interrupción durante la sustitución; no equivale a una transacción de base de datos ni sustituye una copia de seguridad.

Los temporales se limpian en un bloque `finally`. Si no se pueden eliminar, el programa muestra un aviso en lugar de ocultar el error principal.

Servicio: operaciones CRUD

Operación	Algoritmo
Crear archivo	Guardar la cabecera sin productos; pedir confirmación si el archivo ya existe.
Añadir	Cargar, comprobar que el código no exista, insertar en el mapa y guardar.
Listar	Cargar y mostrar los productos junto con el total del inventario.
Modificar	Cargar, comprobar que el código exista, reemplazar el objeto y guardar.
Eliminar	Buscar, pedir confirmación, quitar el código del mapa y guardar.

Se usa `LinkedHashMap<String, Producto>` para localizar los productos por código y conservar su orden de inserción. Añadir datos no consiste en abrir el archivo en modo append: todas las operaciones de cambio reescriben el conjunto completo.

Consola y manejo de errores

El menú utiliza un bloque de texto y un `switch` con flechas. La consola repite las preguntas de cantidad y precio cuando el valor no se puede convertir o es negativo; también acepta `9,90` y lo interpreta como `9.90`.

Al modificar, ENTER conserva el valor actual del campo. Si un nuevo valor es inválido, se cancela esa modificación, se muestra el error y se vuelve al menú sin guardar cambios.

Los errores de validación y de formato se comunican mediante `IllegalArgumentException` o `IllegalStateException`. Los errores de lectura o escritura se envuelven en `UncheckedIOException`, y el menú los muestra sin cerrar la aplicación.

Ejecución en Windows 11

Abra PowerShell en la carpeta del programa y verifique que Java 21 esté disponible. Puede ejecutar el archivo fuente directamente sin compilarlo antes.

```
cd C:\clases\persistencia\programa5-csv
java -version
chcp 65001
java -Dstdout.encoding=UTF-8 Programa5Csv.java
```

Como alternativa, compile y ejecute la clase. No se necesita ninguna opción `-cp` para bibliotecas externas.

```
javac -encoding UTF-8 Programa5Csv.java
java -Dstdout.encoding=UTF-8 Programa5Csv
```

Para utilizar los datos de ejemplo, copie el archivo con el nombre esperado. Hágalo solo si no necesita conservar un `productos.csv` anterior, porque la copia puede reemplazarlo.

```
Copy-Item ejemplo-productos.csv productos.csv -Confirm
java -Dstdout.encoding=UTF-8 Programa5Csv.java
```

Después elija la opción 3 para listar. No elija crear un archivo vacío y confirme su reemplazo si desea conservar los datos importados.

Ejemplo de sesión

En esta sesión se registra un nombre con comas y comillas, se modifica la cantidad y se elimina el producto. Para abreviar, se muestra el menú completo una sola vez.

```
=== PROGRAMA 5: Persistencia en CSV ===

1. Crear archivo (nuevo / vacío)
2. Añadir producto
3. Listar productos
4. Modificar producto
5. Eliminar producto
0. Salir

Opción: 1
Archivo creado: ...\productos.csv

Opción: 2
```

```
Código (sin espacios): p001
Nombre: Aceite "Premium", botella; 1 L
Cantidad (>= 0): 5
Precio (>= 0.00): 9,90
Producto añadido. Total = 49.50

Opción: 4
Código del producto a modificar: P001
Datos actuales: Producto[codigo=P001, nombre=Aceite "Premium", botella; 1 L,
cantidad=5, precio=9.9]
(Presione ENTER para conservar el valor actual)
Nombre [Aceite "Premium", botella; 1 L]:
Cantidad [5]: 8
Precio [9.9]:
Producto modificado. Nuevo total = 79.20

Opción: 5
Código del producto a eliminar: P001
Se eliminará: Producto[codigo=P001, nombre=Aceite "Premium", botella; 1 L,
cantidad=8, precio=9.9]
¿Confirma la eliminación? (s/n): s
Producto eliminado.

Opción: 3
No hay productos registrados.
```

Antes de eliminarlo, la fila persistida después de modificar sería la siguiente. El texto original se recupera sin conservar las comillas adicionales utilizadas por el formato.

```
codigo,nombre,cantidad,precio,total
"P001","Aceite ""Premium"", botella; 1 L",8,9.90,79.20
```

Pruebas de la práctica

El archivo `PruebasPrograma5Csv.java` incluye 41 comprobaciones automáticas y no utiliza JUnit. Crea una carpeta temporal, realiza las operaciones y elimina los datos de prueba al finalizar, sin tocar el `productos.csv` de los estudiantes.

```
javac -encoding UTF-8 Programa5Csv.java PruebasPrograma5Csv.java
java PruebasPrograma5Csv
```

El resultado esperado es `PRUEBAS SUPERADAS: 41 comprobaciones.`. Si alguna condición no se cumple, la prueba lanza `AssertionError` con una descripción.

Caso	Entrada o acción	Resultado esperado
Crear	Archivo inexistente	Se crea solo la cabecera.
Añadir	Nombre con comas, comillas y `;`	Se guarda y se recupera el mismo texto.
Duplicado	`p001` cuando existe `P001`	Se rechaza sin guardar cambios.
Cantidad	`-1` o `abc` en consola	Se vuelve a pedir un entero válido.
Precio	`9,90` en consola	Se acepta y se guarda `9.90`.
Redondeo	Precio `2.555`	El modelo conserva `2.56`.
Modificar	Cantidad nueva y ENTER en el resto	Solo cambia la cantidad y se recalcula el total.
Eliminar	Confirmación `n` o `s`	Se cancela o elimina, respectivamente.
Reapertura	Crear otro repositorio sobre el mismo archivo	Se recuperan los productos persistidos.
Total externo	CSV con total `999`, cantidad 2 y precio 1.50	El total calculado es 3.00.

Caso	Entrada o acción	Resultado esperado
CSV inválido	Comillas abiertas, cabecera incorrecta o columnas incompletas	Se detiene la carga y se informa del error.
Datos inválidos	Cantidad negativa o códigos repetidos en el archivo	No se acepta una carga parcial ni se sobrescribe al añadir.
Campo multilínea	Nombre entre comillas con CRLF interior	El analizador conserva los saltos.
Límites numéricos	`NaN`, infinito o total desbordado	Se rechaza el producto.

Los casos de interacción con el menú complementan las pruebas automáticas de modelo, servicio y repositorio. La confirmación de eliminación se comprueba ejecutando la aplicación y contestando primero `n` y luego `s`.

Importación en una hoja de cálculo y precauciones

Al importar el archivo, seleccione UTF-8, coma como delimitador y comillas dobles como delimitador de texto. Conviene tratar `codigo` como texto para evitar que una herramienta transforme identificadores con ceros iniciales.

Esta práctica usa un dialecto concreto: coma entre columnas y punto decimal. Un archivo exportado con punto y coma como delimitador o con coma decimal no se acepta directamente; debe exportarse con la configuración indicada o adaptarse el lector de forma explícita.

Las comillas CSV no neutralizan fórmulas de una hoja de cálculo. Un código o nombre que empiece por `=`, `+`, `-` o `@` debe revisarse antes de abrir un archivo no confiable en una hoja de cálculo; este programa conserva el texto original y no agrega un prefijo de protección.

Ventajas, límites y comparación

Aspecto	Programa 1: texto delimitado simple	Programa 5: CSV
Separador	Punto y coma.	Coma.
Nombre con separador	El modelo prohíbe `;` en el nombre.	Las comas se conservan dentro de comillas.
Comillas	No interpreta reglas de escape.	Duplica las comillas interiores y las reconstruye.
Registros multilínea	La lectura es por líneas.	El analizador distingue saltos internos y finales de registro.
Lectura defectuosa	Puede ignorar líneas con avisos.	Aborta para evitar una reescritura con pérdida de filas.
Coste de modificación	Reescribe el archivo completo.	También reescribe el archivo completo.

El CSV facilita el intercambio de datos tabulares y no requiere bibliotecas en esta implementación. No impone tipos por sí solo: el programa debe interpretar los textos y validar sus valores.

Esta solución es didáctica y carga el documento completo en memoria. No incluye bloqueo de archivos ni coordinación entre usuarios; dos instancias que escriban al mismo tiempo pueden perder cambios, por lo que debe utilizarse una sola instancia escritora.

Los nombres multilínea importados se conservan, aunque pueden desalinear el listado de consola. En una aplicación mayor se podría mostrar una versión de una sola línea sin alterar el dato persistido.

Ejercicios propuestos

- **Búsqueda por nombre:** añadir una opción que encuentre coincidencias sin distinguir mayúsculas.
- **Filtro de existencias:** exportar a otro CSV solo los productos cuya cantidad sea cero.
- **Separador configurable:** permitir coma o punto y coma manteniendo todas las reglas de comillas.
- **Diagnóstico de errores:** incluir el número de registro al detectar comillas mal formadas.
- **Dinero exacto:** convertir `precio` y `total` a `BigDecimal` y comparar los resultados.
- **Migración:** leer este CSV e insertar los productos con el repositorio JDBC del programa 4.

Preguntas de repaso

1. ¿Por qué un campo con una coma no debe procesarse simplemente con `split(",")`?
2. ¿Cómo se representa una comilla del nombre dentro del CSV?
3. ¿Qué diferencia hay entre un salto de línea dentro de comillas y uno fuera?
4. ¿Por qué se ignora el total almacenado, pero se exige que exista su columna?
5. ¿Qué riesgo evita detener la carga en lugar de omitir registros incorrectos?
6. ¿Qué protege la escritura mediante un temporal y qué problemas no resuelve?
7. ¿Por qué entrecomillar un nombre no basta para evitar fórmulas en una hoja de cálculo?

Código fuente completo

El siguiente código corresponde al archivo independiente `Programa5Csv.java`. Las pruebas automáticas y los datos de ejemplo se entregan como archivos adicionales dentro del proyecto.

```
import java.io.BufferedWriter;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Scanner;

/**
 * PROGRAMA 5 - Persistencia en CSV (Java 21)
 *
 * Gestiona productos (crear archivo, añadir, listar, modificar y eliminar)
 * guardándolos en productos.csv, con comas y comillas dobles.
 *
 * Ejecución (Java 21, sin compilar previamente):
 *     java Programa5Csv.java
 */
public class Programa5Csv {

    // ======================================================================
    // 1. MODELO: record Producto (inmutable, con validación)
    // ======================================================================
    /**
     * Un record genera automáticamente: constructor, getters (codigo(), nombre()...),
     * equals(), hashCode() y toString(). El "constructor compacto" valida los datos.
     */
    record Producto(String codigo, String nombre, int cantidad, double precio) {

        Producto {
            if (codigo == null || !codigo.matches("(?U)\\S+"))
                throw new IllegalArgumentException("El código no puede estar vacío ni contener espacios.");
            if (nombre == null || nombre.isBlank())
                throw new IllegalArgumentException("El nombre no puede estar vacío.");
            if (cantidad < 0)
                throw new IllegalArgumentException("La cantidad debe ser un entero >= 0.");
            if (Double.isNaN(precio) || Double.isInfinite(precio) || precio < 0.0)
                throw new IllegalArgumentException("El precio debe ser un número >= 0.00.");
            codigo = codigo.toUpperCase(Locale.ROOT);   // normalizamos el código
            nombre = nombre.strip();          // quitamos espacios sobrantes
            precio = BigDecimal.valueOf(precio)
                    .setScale(2, RoundingMode.HALF_UP).doubleValue();
            if (!Double.isFinite(cantidad * precio))
                throw new IllegalArgumentException("El total excede el rango de double.");
        }

        /** Campo calculado: NO se pide al usuario, se deriva de cantidad * precio. */
        double total() {
            return cantidad * precio;
        }
    }


    // ======================================================================
    // 2. PERSISTENCIA: CSV con comas, comillas dobles y campos multilínea
    // ======================================================================
    static class RepositorioCsv {
        static final List<String> CABECERA =
```

```
            List.of("codigo", "nombre", "cantidad", "precio", "total");
    private final Path archivo;

    RepositorioCsv(Path archivo) { this.archivo = archivo; }

    boolean existe() { return Files.exists(archivo); }

    /** Crear deja solo la cabecera; el menú confirma si ya hay archivo. */
    void crear() { guardarTodos(List.of()); }

    /**
     * No se omiten registros dañados: abortar evita perderlos en la
     * siguiente reescritura. La columna total se ignora y se recalcula.
     */
    Map<String, Producto> cargarTodos() {
        Map<String, Producto> datos = new LinkedHashMap<>();
        if (!existe()) return datos;
        try {
            String texto = Files.readString(archivo, StandardCharsets.UTF_8);
            // Se tolera el marcador BOM al comienzo de un archivo importado.
            if (texto.startsWith("\uFEFF")) texto = texto.substring(1);
            List<List<String>> filas = analizar(texto);
            if (filas.isEmpty() || !filas.getFirst().equals(CABECERA))
                throw new IllegalStateException("Cabecera CSV ausente o incorrecta.");
            for (int i = 1; i < filas.size(); i++) {
                List<String> c = filas.get(i);
                try {
                    if (c.size() != 5)
                        throw new IllegalArgumentException("Se esperan 5 columnas.");
                    Producto p = new Producto(c.get(0), c.get(1),
                            Integer.parseInt(c.get(2)),
                            Double.parseDouble(c.get(3)));
                    if (datos.putIfAbsent(p.codigo(), p) != null)
                        throw new IllegalArgumentException("Código duplicado: " + p.codigo());
                } catch (IllegalArgumentException e) {
                    throw new IllegalStateException("Registro CSV " + (i + 1)
                            + " inválido: " + e.getMessage(), e);
                }
            }
            return datos;
        } catch (IOException e) {
            throw new UncheckedIOException("No se pudo leer " + archivo, e);
        }
    }

    /**
     * Guardar no significa añadir bytes al final: se reescribe el conjunto.
     * El temporal único se crea en la misma carpeta del archivo definitivo.
     */
    void guardarTodos(Iterable<Producto> productos) {
        Path temporal = null;
        try {
            Path destino = archivo.toAbsolutePath();
            temporal = Files.createTempFile(destino.getParent(), "productos-", ".tmp");
            try (BufferedWriter w = Files.newBufferedWriter(
                    temporal, StandardCharsets.UTF_8)) {
                w.write(String.join(",", CABECERA) + "\r\n");
                for (Producto p : productos) {
                    w.write(escapar(p.codigo()) + "," + escapar(p.nombre()) + ","
                            + p.cantidad() + ","
                            + String.format(Locale.ROOT, "%.2f", p.precio()) + ","
                            + String.format(Locale.ROOT, "%.2f", p.total()) + "\r\n");
                }
            }
            try {
                Files.move(temporal, destino, StandardCopyOption.REPLACE_EXISTING,
                        StandardCopyOption.ATOMIC_MOVE);
            } catch (AtomicMoveNotSupportedException e) {
                // Este respaldo NO conserva la garantía de sustitución atómica.
                Files.move(temporal, destino, StandardCopyOption.REPLACE_EXISTING);
            }
        } catch (IOException e) {
            throw new UncheckedIOException("No se pudo guardar " + archivo, e);
        } finally {
            if (temporal != null) {
                try { Files.deleteIfExists(temporal); }
```

```
                catch (IOException e) {
                    System.err.println("Aviso: no se pudo limpiar " + temporal);
                }
            }
        }
    }

    /** Todo texto se entrecomilla; una comilla interior se duplica. */
    static String escapar(String valor) {
        return "\"" + valor.replace("\"", "\"\"") + "\"";
    }

    /**
     * Analizador por estados: una coma o un salto de línea solo separa
     * cuando está FUERA de un campo entrecomillado. No usar split(",").
     * Acepta CRLF o LF; no admite espacios después de una comilla de cierre.
     */
    static List<List<String>> analizar(String texto) {
        List<List<String>> filas = new ArrayList<>();
        List<String> fila = new ArrayList<>();
        StringBuilder campo = new StringBuilder();
        boolean entreComillas = false;
        boolean cerrado = false;
        boolean iniciado = false;

        for (int i = 0; i < texto.length(); i++) {
            char ch = texto.charAt(i);
            if (entreComillas) {
                if (ch == '"') {
                    if (i + 1 < texto.length() && texto.charAt(i + 1) == '"') {
                        campo.append('"');
                        i++; // dos comillas en CSV representan una en el dato
                    } else {
                        entreComillas = false;
                        cerrado = true;
                    }
                } else campo.append(ch); // conserva comas y saltos interiores
            } else if (ch == ',' || ch == '\r' || ch == '\n') {
                fila.add(campo.toString());
                campo.setLength(0);
                iniciado = false;
                cerrado = false;
                if (ch != ',') {
                    filas.add(fila);
                    fila = new ArrayList<>();
                    if (ch == '\r' && i + 1 < texto.length()
                            && texto.charAt(i + 1) == '\n') i++;
                }
            } else if (cerrado) {
                throw new IllegalStateException(
                        "CSV inválido: carácter después de comilla de cierre.");
            } else if (ch == '"') {
                if (iniciado)
                    throw new IllegalStateException(
                            "CSV inválido: comilla en campo sin entrecomillar.");
                entreComillas = true;
                iniciado = true;
            } else {
                campo.append(ch);
                iniciado = true;
            }
        }
        if (entreComillas)
            throw new IllegalStateException("CSV inválido: comillas sin cerrar.");
        // No crear un registro vacío ficticio después del último CRLF.
        if (iniciado || cerrado || !fila.isEmpty()) {
            fila.add(campo.toString());
            filas.add(fila);
        }
        return filas;
    }
}

// ============================================================
// 3. SERVICIO: operaciones CRUD sobre el repositorio
// ============================================================
```

```java
static class ServicioProductos {
    private final RepositorioCsv repo;

    ServicioProductos(RepositorioCsv repo) {
        this.repo = repo;
    }

    void crearArchivo() { repo.crear(); }

    boolean existeArchivo() { return repo.existe(); }

    List<Producto> listar() { return new ArrayList<>(repo.cargarTodos().values()); }

    Optional<Producto> buscar(String codigo) {
        return Optional.ofNullable(repo.cargarTodos().get(codigo.toUpperCase(Locale.ROOT)));
    }

    /** AÑADIR: falla si el código ya existe (clave única). */
    void anadir(Producto p) {
        Map<String, Producto> datos = repo.cargarTodos();
        if (datos.containsKey(p.codigo()))
            throw new IllegalStateException("Ya existe un producto con código " + p.codigo());
        datos.put(p.codigo(), p);
        repo.guardarTodos(datos.values());
    }

    /** MODIFICAR: reemplaza el producto con el mismo código. */
    void modificar(Producto p) {
        Map<String, Producto> datos = repo.cargarTodos();
        if (!datos.containsKey(p.codigo()))
            throw new IllegalStateException("No existe el producto " + p.codigo());
        datos.put(p.codigo(), p);   // put sobre clave existente = reemplazo, conserva orden
        repo.guardarTodos(datos.values());
    }

    /** ELIMINAR: devuelve true si se eliminó. */
    boolean eliminar(String codigo) {
        Map<String, Producto> datos = repo.cargarTodos();
        if (datos.remove(codigo.toUpperCase(Locale.ROOT)) == null) return false;
        repo.guardarTodos(datos.values());
        return true;
    }
}

// ============================================================================
// 4. CONSOLA: lectura validada de datos
// ============================================================================
static class Consola {
    /** Señal para terminar si se cierra la entrada estándar (Ctrl+D / Ctrl+Z). */
    static class FinDeEntrada extends RuntimeException {
        private static final long serialVersionUID = 1L;
    }

    private final Scanner sc = new Scanner(System.in, StandardCharsets.UTF_8);

    String leerLinea(String mensaje) {
        System.out.print(mensaje);
        if (!sc.hasNextLine()) throw new FinDeEntrada();
        return sc.nextLine().strip();
    }

    String leerCodigo(String mensaje) {
        while (true) {
            String s = leerLinea(mensaje);
            if (s.matches("(?U)\\S+")) return s.toUpperCase(Locale.ROOT);
            System.out.println("  Código inválido: no debe estar vacío ni tener espacios.");
        }
    }

    String leerNombre(String mensaje) {
        while (true) {
            String s = leerLinea(mensaje);
            if (s.isBlank()) System.out.println("  El nombre no puede estar vacío.");
            else return s;
        }
    }
```

```
    int leerEnteroNoNegativo(String mensaje) {
        while (true) {
            String s = leerLinea(mensaje);
            try {
                int n = Integer.parseInt(s);
                if (n >= 0) return n;
            } catch (NumberFormatException ignored) { }
            System.out.println("  Ingrese un número entero mayor o igual a 0.");
        }
    }

    double leerDecimalNoNegativo(String mensaje) {
        while (true) {
            String s = leerLinea(mensaje).replace(',', '.'); // acepta 4,50 o 4.50
            try {
                double d = Double.parseDouble(s);
                if (d >= 0.0 && Double.isFinite(d)) return d;
            } catch (NumberFormatException ignored) { }
            System.out.println("  Ingrese un número decimal mayor o igual a 0.00.");
        }
    }

    boolean confirmar(String mensaje) {
        String s = leerLinea(mensaje + " (s/n): ");
        return s.equalsIgnoreCase("s") || s.equalsIgnoreCase("si") || s.equalsIgnoreCase("sí");
    }
}

// ========================================================================
// 5. PROGRAMA PRINCIPAL (menú)
// ========================================================================
private static final Path ARCHIVO = Path.of("productos.csv");

public static void main(String[] args) {
    var servicio = new ServicioProductos(new RepositorioCsv(ARCHIVO));
    var consola = new Consola();

    System.out.println("=== PROGRAMA 5: Persistencia en CSV ===");
    System.out.println("Archivo: " + ARCHIVO.toAbsolutePath());

    try {
        boolean salir = false;
        while (!salir) {
            mostrarMenu();
            String opcion = consola.leerLinea("Opción: ");
            try {
                // switch con flechas (Java 14+): sin "break", sin caída accidental
                switch (opcion) {
                    case "1" -> crear(servicio, consola);
                    case "2" -> anadir(servicio, consola);
                    case "3" -> listar(servicio);
                    case "4" -> modificar(servicio, consola);
                    case "5" -> eliminar(servicio, consola);
                    case "0" -> salir = true;
                    default -> System.out.println("Opción no válida.");
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (UncheckedIOException e) {
                System.out.println("Error de E/S: " + e.getMessage());
            }
        }
    } catch (Consola.FinDeEntrada e) {
        System.out.println();
    }
    System.out.println("Fin del programa.");
}

private static void mostrarMenu() {
    System.out.println("""

            ---------- MENÚ ----------
            1. Crear archivo (nuevo / vacío)
            2. Añadir producto
            3. Listar productos
```

```
                4. Modificar producto
                5. Eliminar producto
                0. Salir
                --------------------------""");
    }

    private static void crear(ServicioProductos s, Consola c) {
        if (s.existeArchivo() && !c.confirmar("El archivo ya existe. ¿Borrar todo su contenido?")) {
            System.out.println("Operación cancelada.");
            return;
        }
        s.crearArchivo();
        System.out.println("Archivo creado: " + ARCHIVO.toAbsolutePath());
    }

    private static void anadir(ServicioProductos s, Consola c) {
        String codigo = c.leerCodigo("Código (sin espacios): ");
        if (s.buscar(codigo).isPresent()) {
            System.out.println("Ya existe un producto con código " + codigo);
            return;
        }
        String nombre = c.leerNombre("Nombre: ");
        int cantidad = c.leerEnteroNoNegativo("Cantidad (>= 0): ");
        double precio = c.leerDecimalNoNegativo("Precio (>= 0.00): ");
        var p = new Producto(codigo, nombre, cantidad, precio);
        s.anadir(p);
        System.out.printf("Producto añadido. Total = %.2f%n", p.total());
    }

    private static void listar(ServicioProductos s) {
        List<Producto> lista = s.listar();
        if (lista.isEmpty()) {
            System.out.println("No hay productos registrados.");
            return;
        }
        String formato = "%-10s %-30s %10s %12s %14s%n";
        System.out.printf(formato, "CÓDIGO", "NOMBRE", "CANTIDAD", "PRECIO", "TOTAL");
        System.out.println("-".repeat(80));
        double granTotal = 0;
        for (Producto p : lista) {
            System.out.printf("%-10s %-30s %10d %12.2f %14.2f%n",
                    p.codigo(), p.nombre(), p.cantidad(), p.precio(), p.total());
            granTotal += p.total();
        }
        System.out.println("-".repeat(80));
        System.out.printf("%-10s %-30s %10s %12s %14.2f%n", "", "TOTAL INVENTARIO", "", "", granTotal);
    }

    private static void modificar(ServicioProductos s, Consola c) {
        String codigo = c.leerCodigo("Código del producto a modificar: ");
        Optional<Producto> encontrado = s.buscar(codigo);
        if (encontrado.isEmpty()) {
            System.out.println("No existe el producto " + codigo);
            return;
        }
        Producto actual = encontrado.get();
        System.out.println("Datos actuales: " + actual);
        System.out.println("(Presione ENTER para conservar el valor actual)");

        String nombre = c.leerLinea("Nombre [" + actual.nombre() + "]: ");
        String cant = c.leerLinea("Cantidad [" + actual.cantidad() + "]: ");
        String prec = c.leerLinea("Precio [" + actual.precio() + "]: ").replace(',', '.');

        // Si el usuario deja vacío, se conserva el valor; si no, el record valida el nuevo valor.
        var nuevo = new Producto(
                actual.codigo(),
                nombre.isEmpty() ? actual.nombre() : nombre,
                cant.isEmpty() ? actual.cantidad() : parsearEntero(cant),
                prec.isEmpty() ? actual.precio() : parsearDecimal(prec));
        s.modificar(nuevo);
        System.out.printf("Producto modificado. Nuevo total = %.2f%n", nuevo.total());
    }

    private static void eliminar(ServicioProductos s, Consola c) {
        String codigo = c.leerCodigo("Código del producto a eliminar: ");
        Optional<Producto> p = s.buscar(codigo);
```

```
        if (p.isEmpty()) {
            System.out.println("No existe el producto " + codigo);
            return;
        }
        System.out.println("Se eliminará: " + p.get());
        if (c.confirmar("¿Confirma la eliminación?") && s.eliminar(codigo))
            System.out.println("Producto eliminado.");
        else
            System.out.println("Operación cancelada.");
    }

    private static int parsearEntero(String s) {
        try {
            return Integer.parseInt(s);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("La cantidad debe ser un entero >= 0.");
        }
    }

    private static double parsearDecimal(String s) {
        try {
            return Double.parseDouble(s);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("El precio debe ser un número >= 0.00.");
        }
    }
}
```